```
#!/usr/bin/env python3
# notify_on_mqtt_dummy.py
# Compact runnable example that listens to MQTT and tries to send SMS (Twilio) and email (SMTP).
# Contains randomized/dummy contact values and is safe to run (errors are handled).

import time
import logging
import smtplib
import random
from email.message import EmailMessage

import paho.mqtt.client as mqtt

# Optional Twilio import: wrap in try in case not installed or creds invalid
try:
    from twilio.rest import Client as TwilioClient
    TWILIO_AVAILABLE = True
except Exception:
    TWILIO_AVAILABLE = False

# ========== DUMMY CONFIG (replace with real values as needed) ==========
# MQTT (public test broker used here so script can connect/run)
MQTT_BROKER = "test.mosquitto.org"
MQTT_PORT = 1883
MQTT_TOPIC = "edge/chromatography/alert/test"  # example topic

# Twilio dummy credentials (these are fake/random; SMS attempt will likely fail but is caught)
TWILIO_ACCOUNT_SID = "AC" + "".join(random.choices("0123456789abcdef", k=32))
TWILIO_AUTH_TOKEN = "".join(random.choices("0123456789abcdef", k=32))
TWILIO_FROM_NUMBER = "+15005550006"   # Twilio test number format (won't actually send)
TECH_SMS_NUMBER = "+27831234567"      # random South African-style number

# SMTP dummy settings (no real login)
SMTP_HOST = "smtp.gmail.com"
SMTP_PORT = 587
SMTP_USER = "dummy.user@example.com"
SMTP_PASS = "DummyPassword123!"
ESKOM_EMAIL = "alerts@eskom.co.za"    # example Eskom address
EMAIL_FROM = "edge-device@example.com"

# Retry/config
SMS_RETRY = 2
EMAIL_RETRY = 2
#
==========================================================================
```

```
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s:
%(message)s")

# Initialize Twilio client if available
twilio_client = None
if TWILIO_AVAILABLE:
    try:
        twilio_client = TwilioClient(TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN)
        logging.info("Twilio library loaded; client created (with dummy creds).")
    except Exception as e:
        logging.warning("Could not initialize Twilio client: %s", e)
        twilio_client = None
else:
    logging.info("Twilio library not installed or unavailable. SMS attempts will be logged only.")

def send_sms(body):
    """
    Attempt to send SMS via Twilio if client present; otherwise just log the message.
    Returns True on success (or simulated success), False on failure.
    """
    if twilio_client is None:
        logging.info("SMS (simulated) to %s: %s", TECH_SMS_NUMBER, body)
        return True

    for attempt in range(1, SMS_RETRY + 1):
        try:
            msg = twilio_client.messages.create(
                body=body,
                from_=TWILIO_FROM_NUMBER,
                to=TECH_SMS_NUMBER
            )
            logging.info("SMS sent, sid=%s", getattr(msg, "sid", "N/A"))
            return True
        except Exception as e:
            logging.warning("SMS send attempt %d failed: %s", attempt, e)
            time.sleep(1)
    logging.error("Failed to send SMS after %d attempts (dummy creds likely invalid)",
SMS_RETRY)
    return False

def send_email(subject, body):
    """
    Send email via configured SMTP server. Uses TLS. Errors are caught and retried.
    """
    msg = EmailMessage()
    msg["From"] = EMAIL_FROM
    msg["To"] = ESKOM_EMAIL
    msg["Subject"] = subject
```

```
    msg.set_content(body)

    for attempt in range(1, EMAIL_RETRY + 1):
        try:
            with smtplib.SMTP(SMTP_HOST, SMTP_PORT, timeout=15) as s:
                s.ehlo()
                s.starttls()
                s.ehlo()
                # Attempt login only if non-empty credentials
                if SMTP_USER and SMTP_PASS:
                    try:
                        s.login(SMTP_USER, SMTP_PASS)
                    except Exception as login_err:
                        # Login may fail for dummy creds; still allow sending if server permits
                        logging.warning("SMTP login failed (attempt %d): %s", attempt, login_err)
                s.send_message(msg)
            logging.info("Email sent (or attempted) to %s", ESKOM_EMAIL)
            return True
        except Exception as e:
            logging.warning("Email send attempt %d failed: %s", attempt, e)
            time.sleep(1)
    logging.error("Failed to send email after %d attempts", EMAIL_RETRY)
    return False

def handle_payload(payload):
    """
    Simple payload handler. Decodes bytes to text and composes notification messages.
    """
    text = payload.decode("utf-8", errors="replace")
    logging.info("Received payload: %s", text)

    # Compose messages (include a timestamp and random event id for uniqueness)
    timestamp = time.strftime("%Y-%m-%d %H:%M:%S")
    event_id = "".join(random.choices("ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789",
k=6))
    sms_body = f"ALERT [{event_id}] {timestamp}: {text}"
    email_subject = f"Edge Chromatography Alert [{event_id}]"
    email_body = f"Alert ID: {event_id}\nTime: {timestamp}\n\nPayload:\n{text}\n"

    send_sms(sms_body)
    send_email(email_subject, email_body)

# MQTT callbacks
def on_connect(client, userdata, flags, rc):
    if rc == 0:
        logging.info("Connected to MQTT broker %s:%d", MQTT_BROKER, MQTT_PORT)
        client.subscribe(MQTT_TOPIC, qos=1)
        logging.info("Subscribed to topic: %s", MQTT_TOPIC)
```

```
    else:
        logging.error("MQTT connect failed with rc=%s", rc)

def on_message(client, userdata, msg):
    logging.info("MQTT message received on %s (len=%d)", msg.topic, len(msg.payload))
    try:
        handle_payload(msg.payload)
    except Exception as e:
        logging.exception("Error handling payload: %s", e)

def main():
    client = mqtt.Client(client_id=f"edge-notify-{random.randint(1000,9999)}")
    client.on_connect = on_connect
    client.on_message = on_message

    try:
        client.connect(MQTT_BROKER, MQTT_PORT, keepalive=60)
    except Exception as e:
        logging.error("Could not connect to MQTT broker %s:%d - %s", MQTT_BROKER,
MQTT_PORT, e)
        return

    try:
        logging.info("Starting MQTT loop. Publish to topic '%s' on broker '%s' to test.",
MQTT_TOPIC, MQTT_BROKER)
        client.loop_forever()
    except KeyboardInterrupt:
        logging.info("Interrupted by user, exiting.")
    except Exception as e:
        logging.exception("MQTT loop terminated unexpectedly: %s", e)

if __name__ == "__main__":
    main()
```